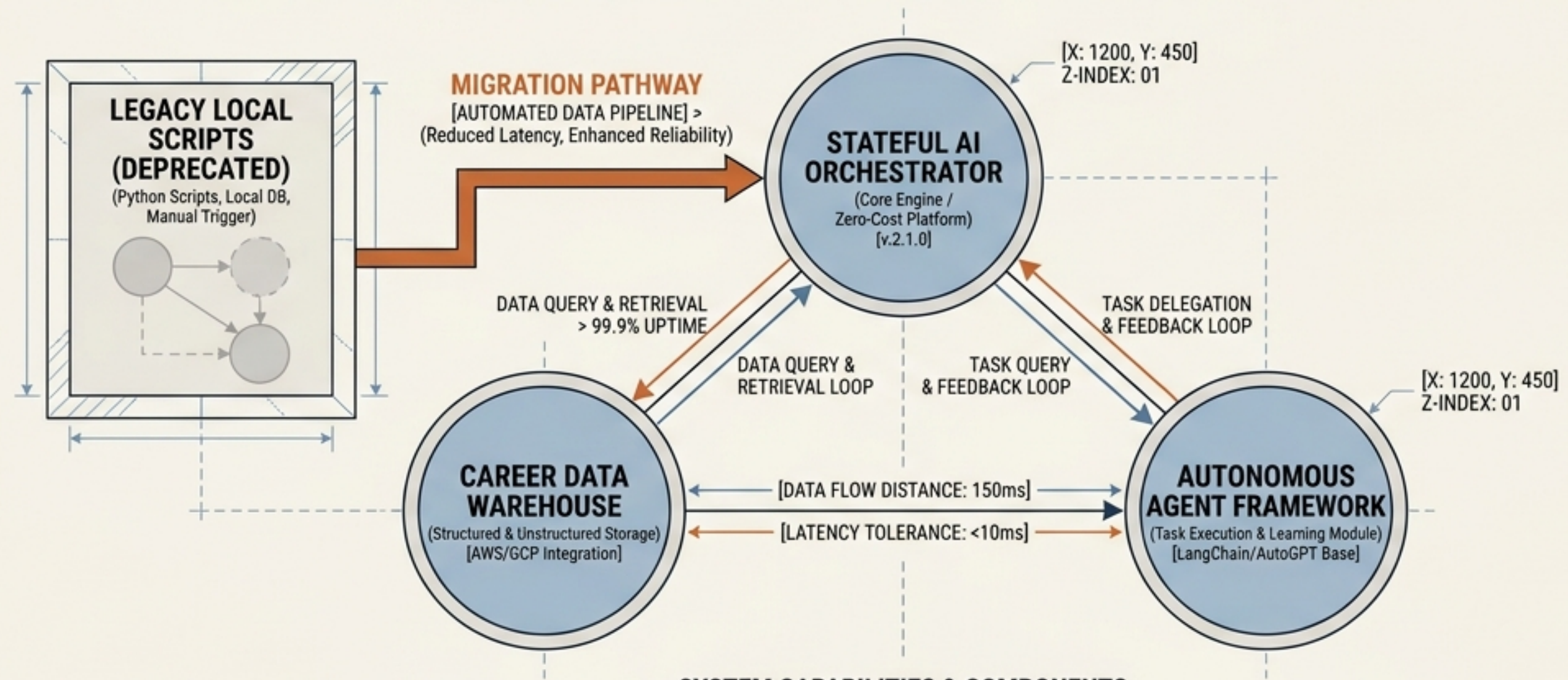


ARCHITECTING THE AUTONOMOUS CAREER AGENT

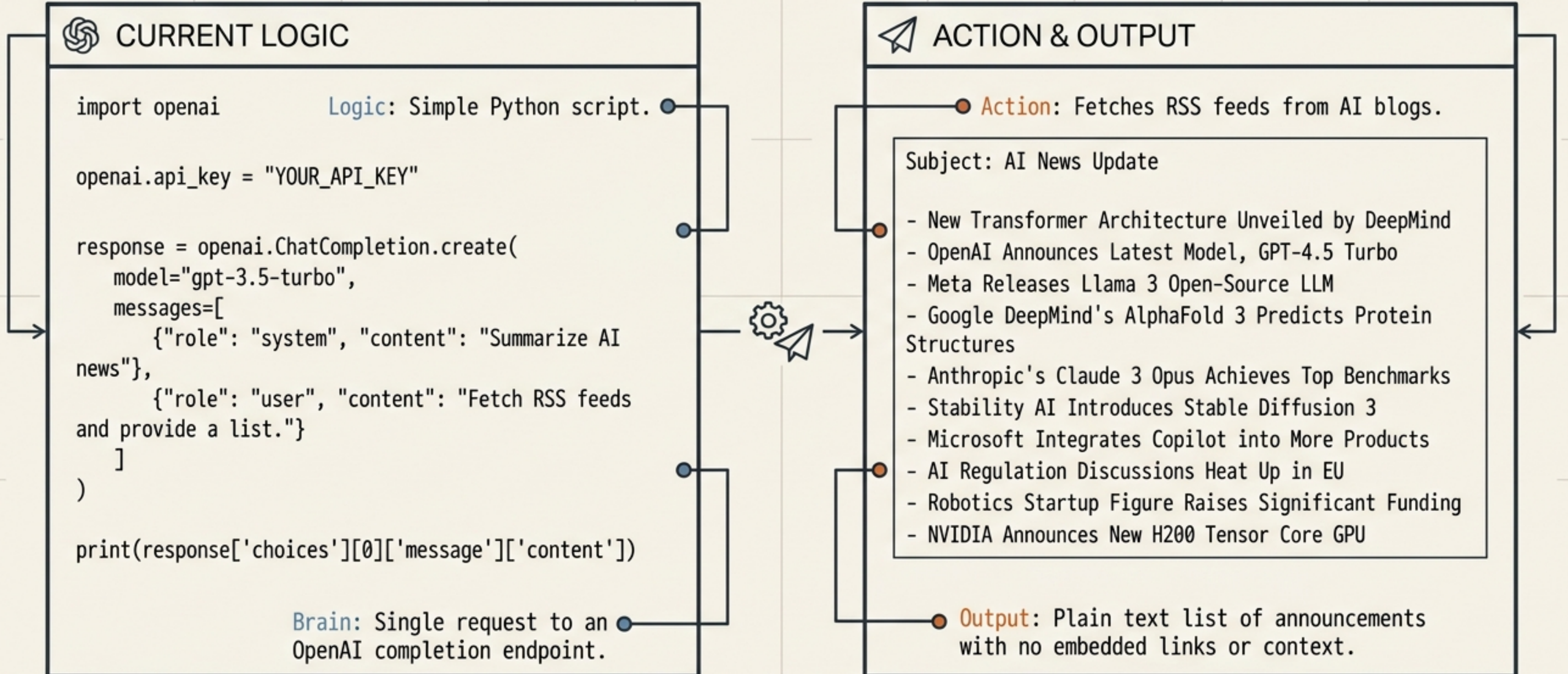
Migrating from Local Scripts to a Zero-Cost, Stateful AI Orchestrator



SYSTEM CAPABILITIES & COMPONENTS

ZERO-COST INFRASTRUCTURE		STATEFUL AI OPERATIONS	
  	> SERVERLESS COMPUTE (e.g., Cloud Functions, Lambda)	  	> CONTINUOUS LEARNING (Contextual Memory)
	> FREE-TIER STORAGE (e.g., MongoDB Atlas, Firebase)		> AUTONOMOUS DECISION MAKING (Policy-Based Actions)
	> ZERO-COST APIs & WEBHOOKS		> SECURE DATA HANDLING (Encryption & Access Control)

The Starting Point: Linear Automation



The “Always On” Infrastructure Problem

LOCAL AUTOMATION (PROBLEM STATE)



Core Insight: The script currently relies on a sleeping laptop to trigger its 8:00 AM run.

CLOUD ORCHESTRATION (PRODUCTION STATE) ←



The Mandate: Moving from Local Automation to Cloud Orchestration to build a production-grade asset.

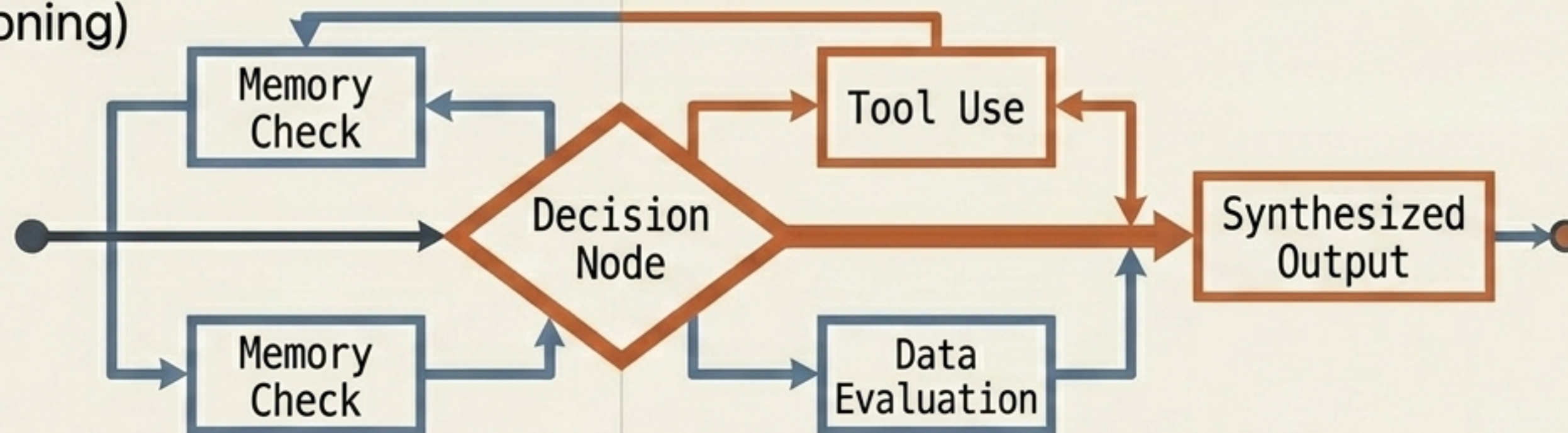
Defining the Shift: Apps vs. Agents

The App (Linear)



Follows a rigid A → B → C path. Executes commands without context.

The Agent (Reasoning)



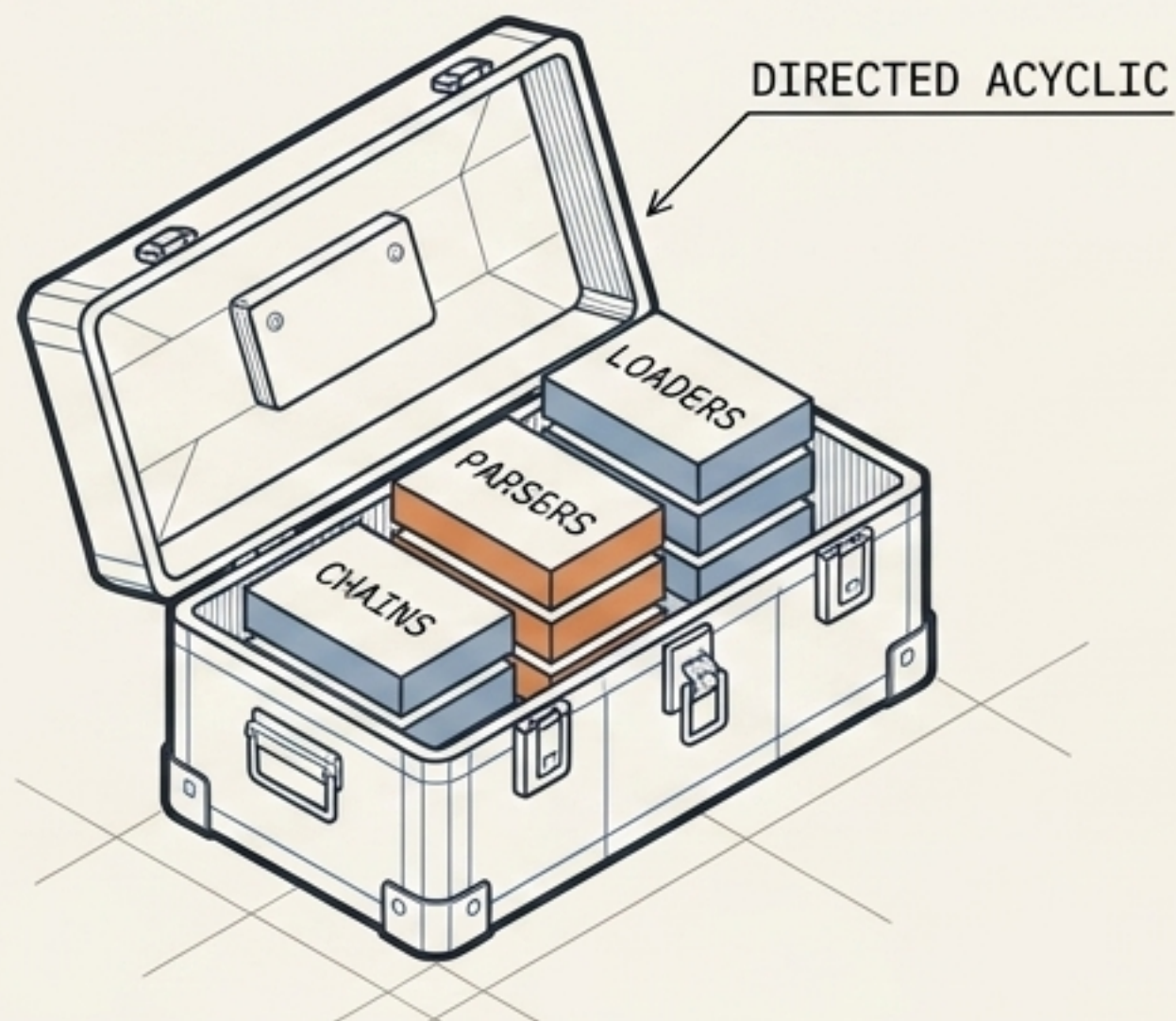
Uses a state machine. Evaluates data, decides to use tools, and navigates loops before synthesizing an output.

Choosing the Execution Environment

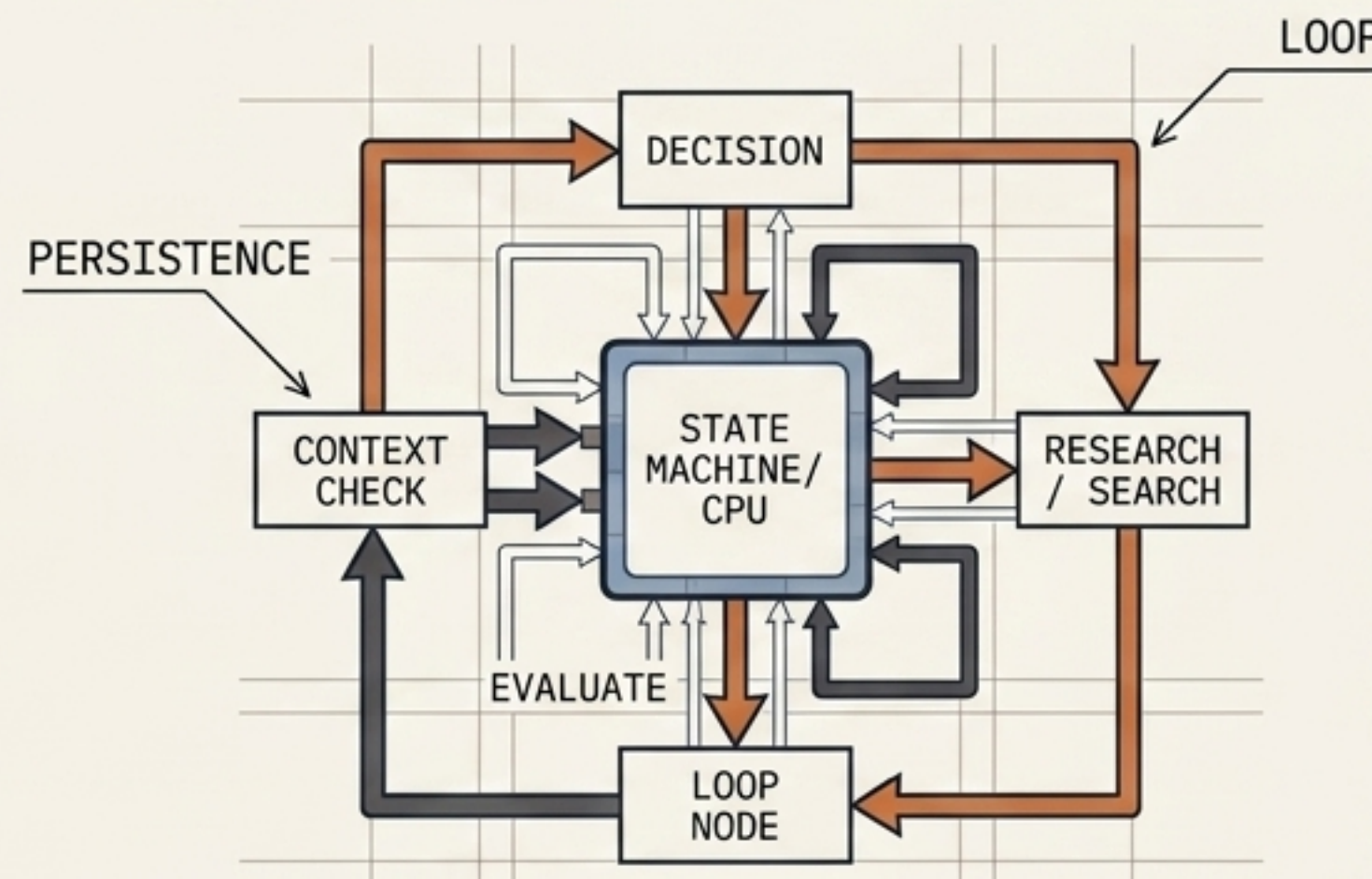
	n8n (Low-Code)	LangGraph (Pro-Code)
Logic	Visual drag-and-drop nodes.	Python-based state machines (cycles/loops).
Agentic Behavior	Tool Use (If X, send to Slack)	Reasoning (Summarize, then evaluate against user career goals)
UI Capability	Limited custom UI.	Pairs seamlessly with Streamlit or Chainlit.

Verdict: LangGraph is the 2026 industry standard for Forward Deployed Engineers.

The Core Misconception: LangChain vs. LangGraph



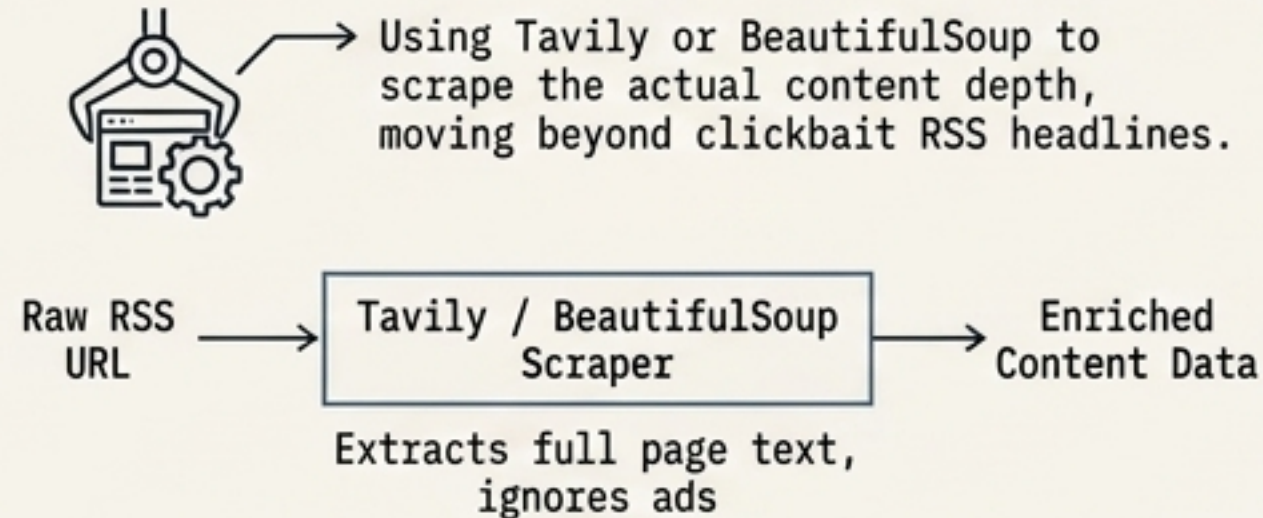
LangChain (The Library): The connectors. Grabs RSS data, parses outputs, and links steps. Struggles with recursive loops.



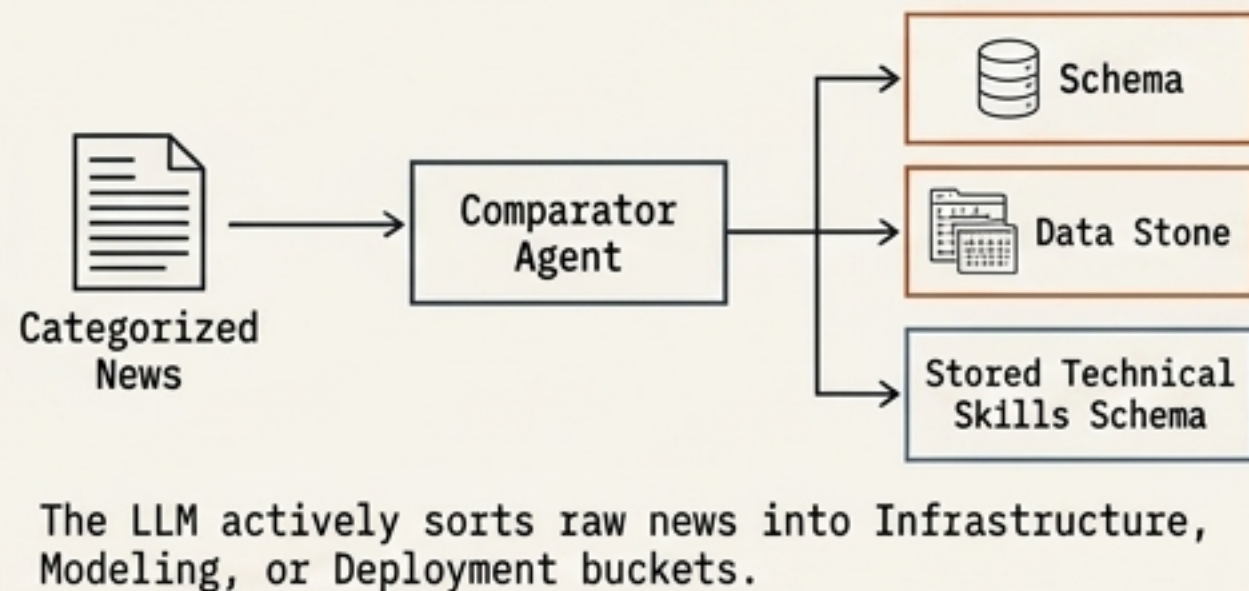
LangGraph (The Orchestrator): The state machine. Built on top of LangChain. Enables the agent to research a topic, realize it needs more context, and loop back to search again.

Engineering the Reasoning Step

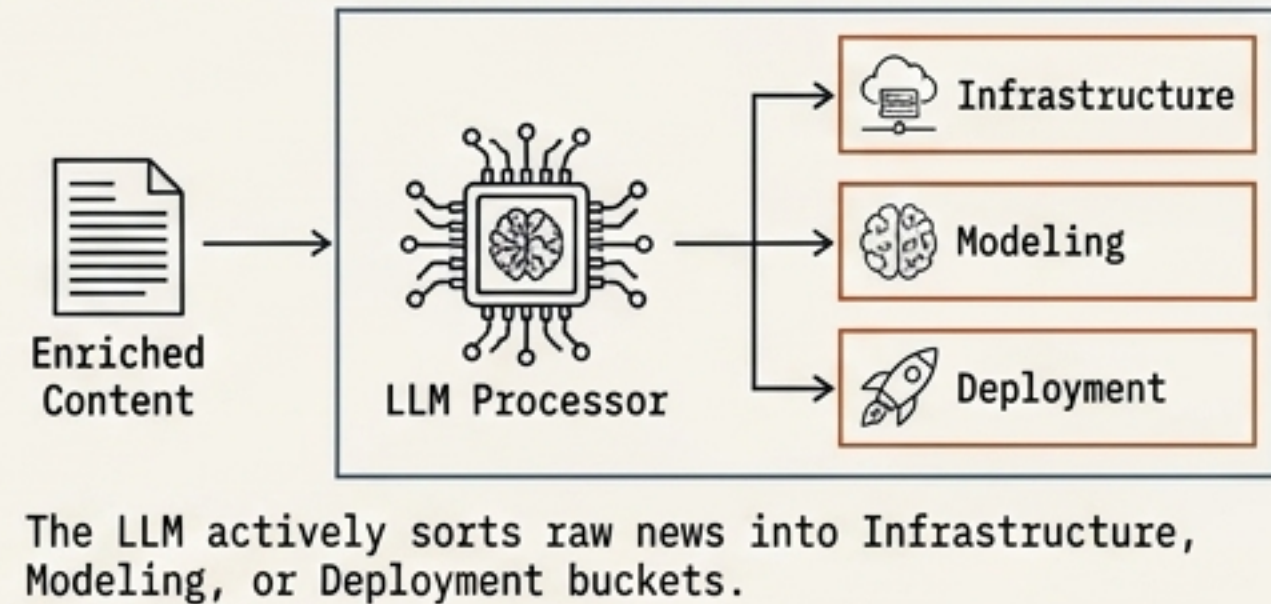
Step 1: Link Enrichment



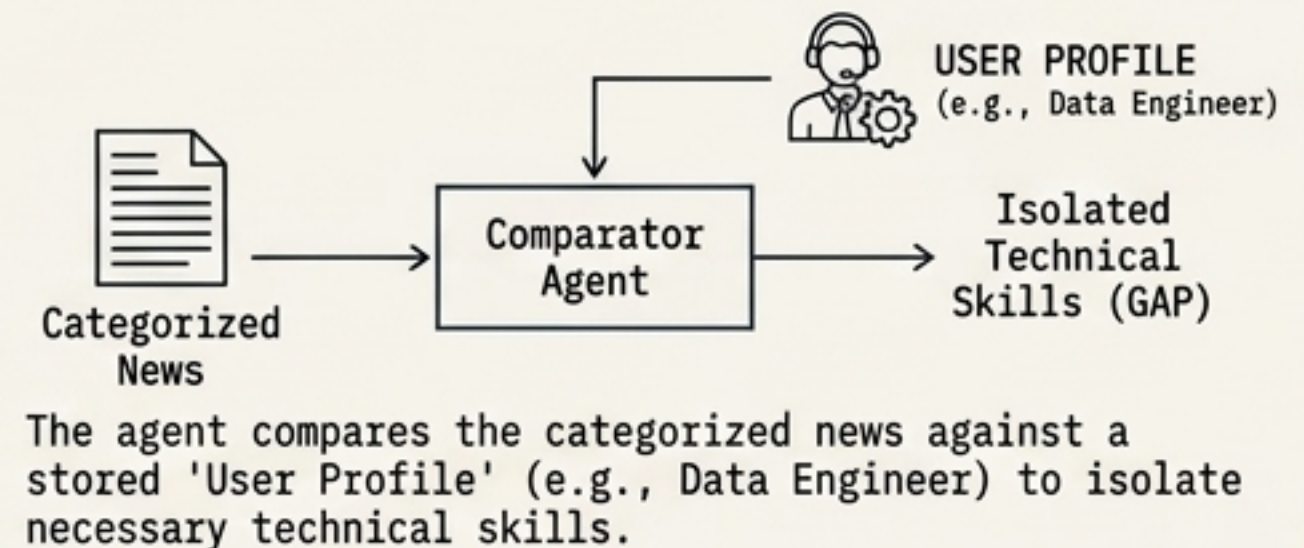
Step 2: Categorization



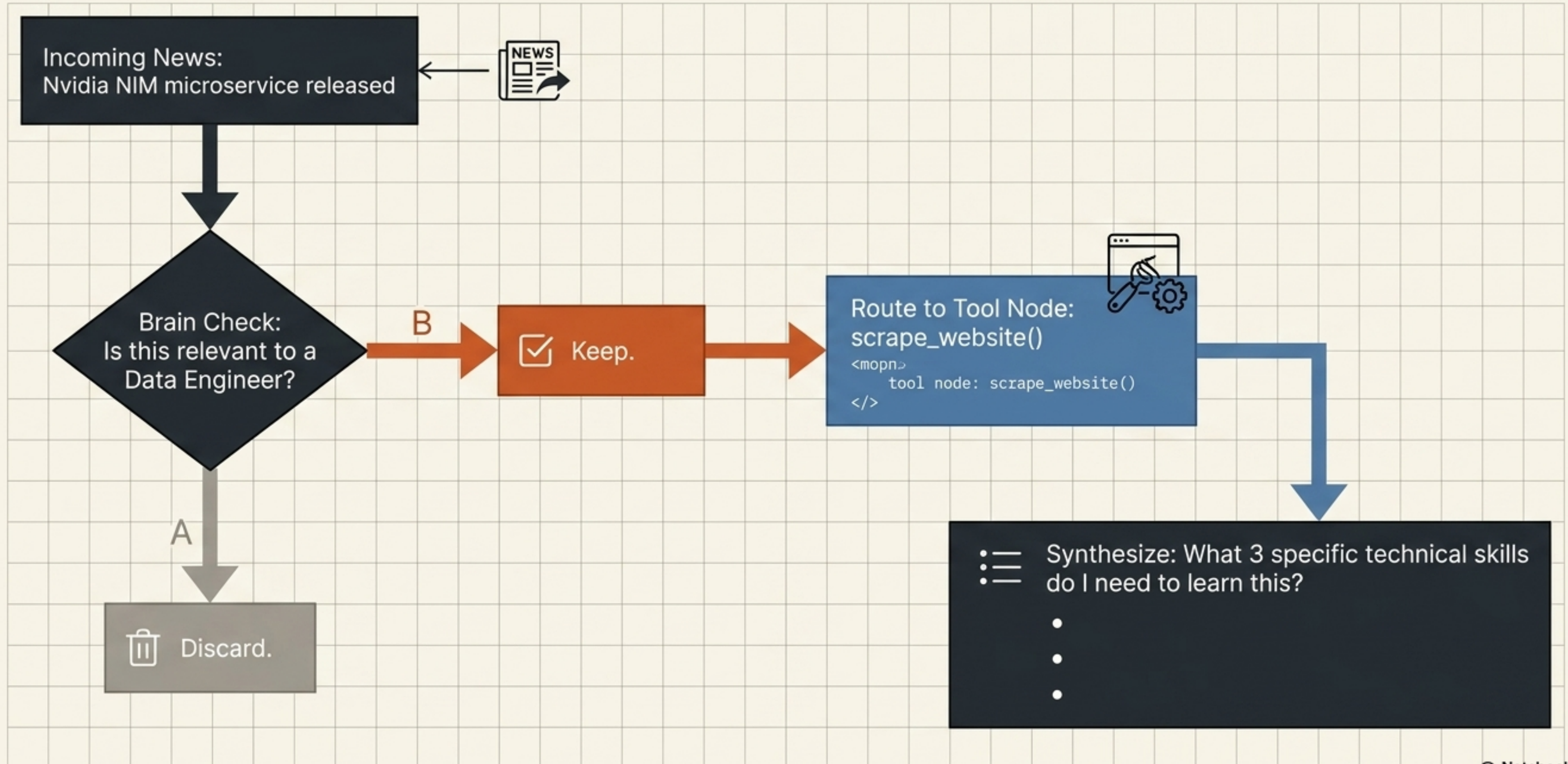
Step 2: Categorization



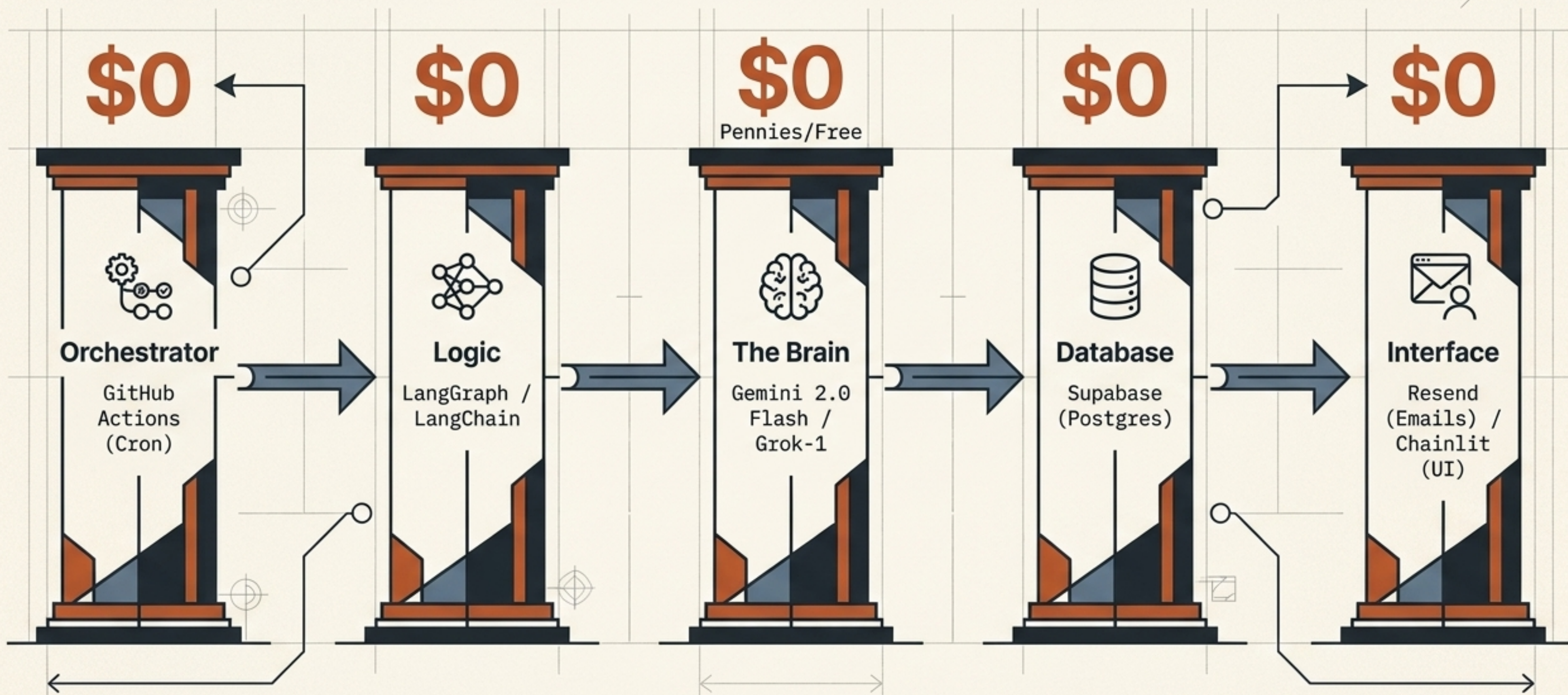
Step 3: Gap Analysis



The Agentic Workflow in Action

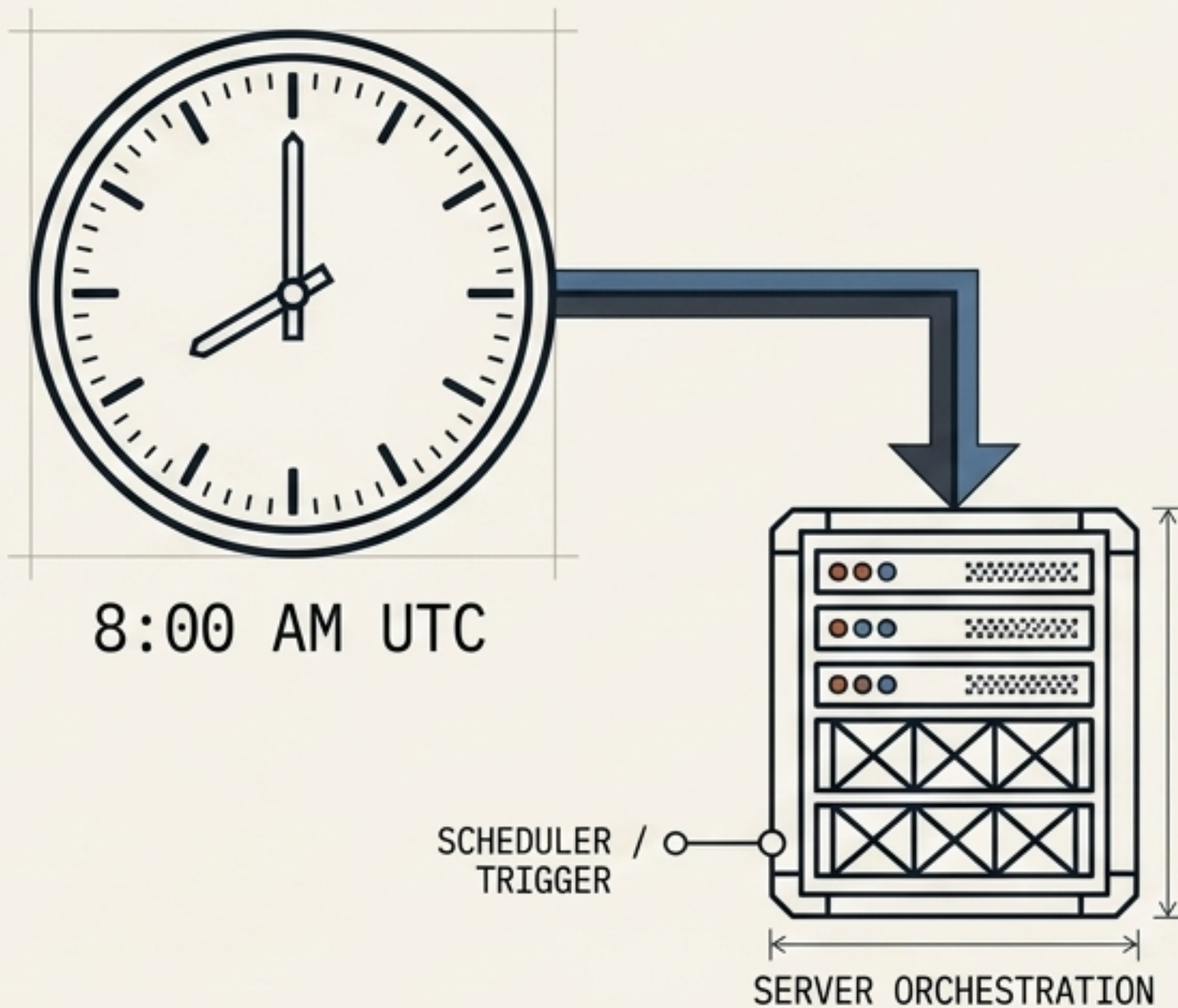


The 2026 Zero-Dollar Architecture

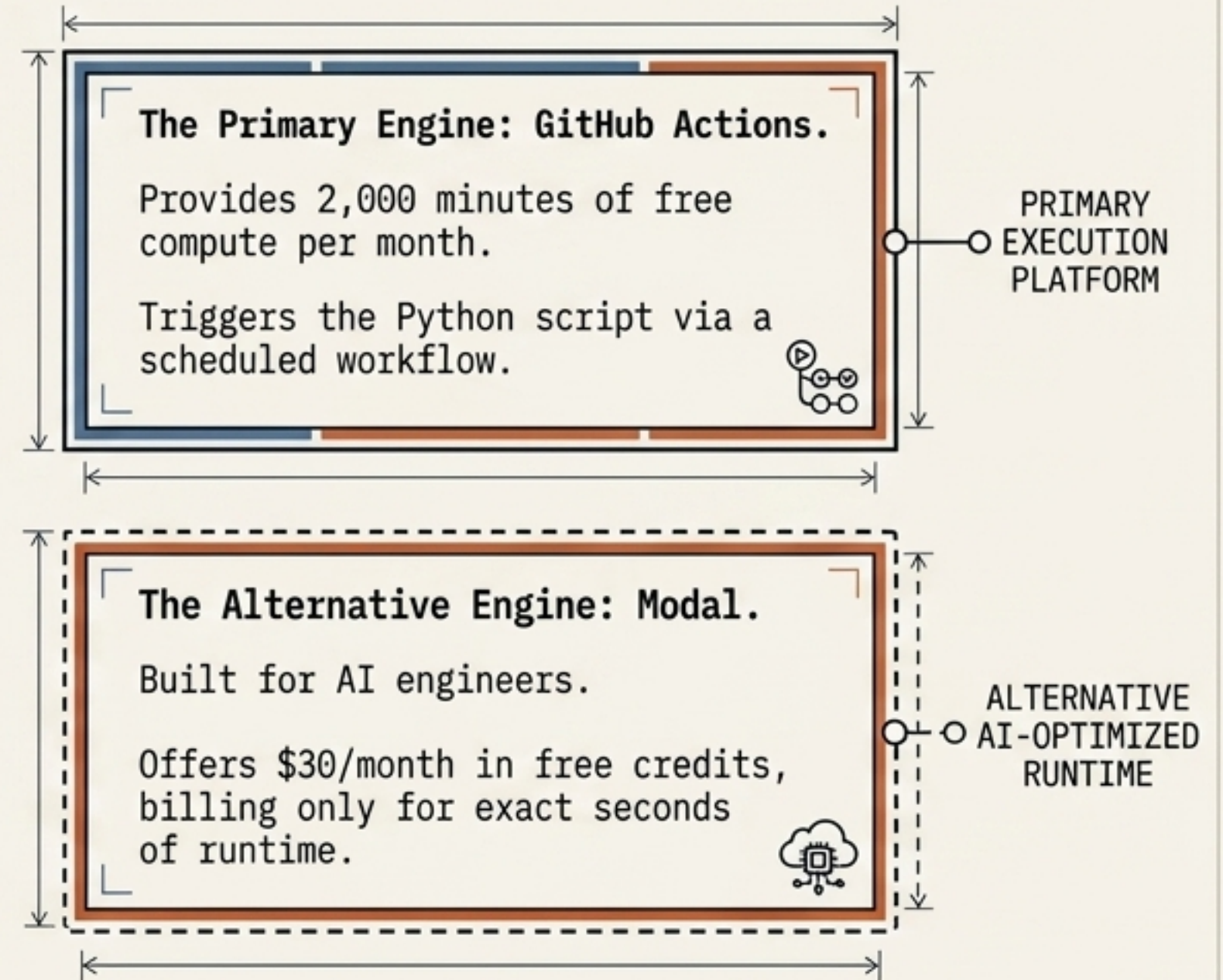


Layer 1: Zero-Cost Cloud Orchestration

Visual Schematic



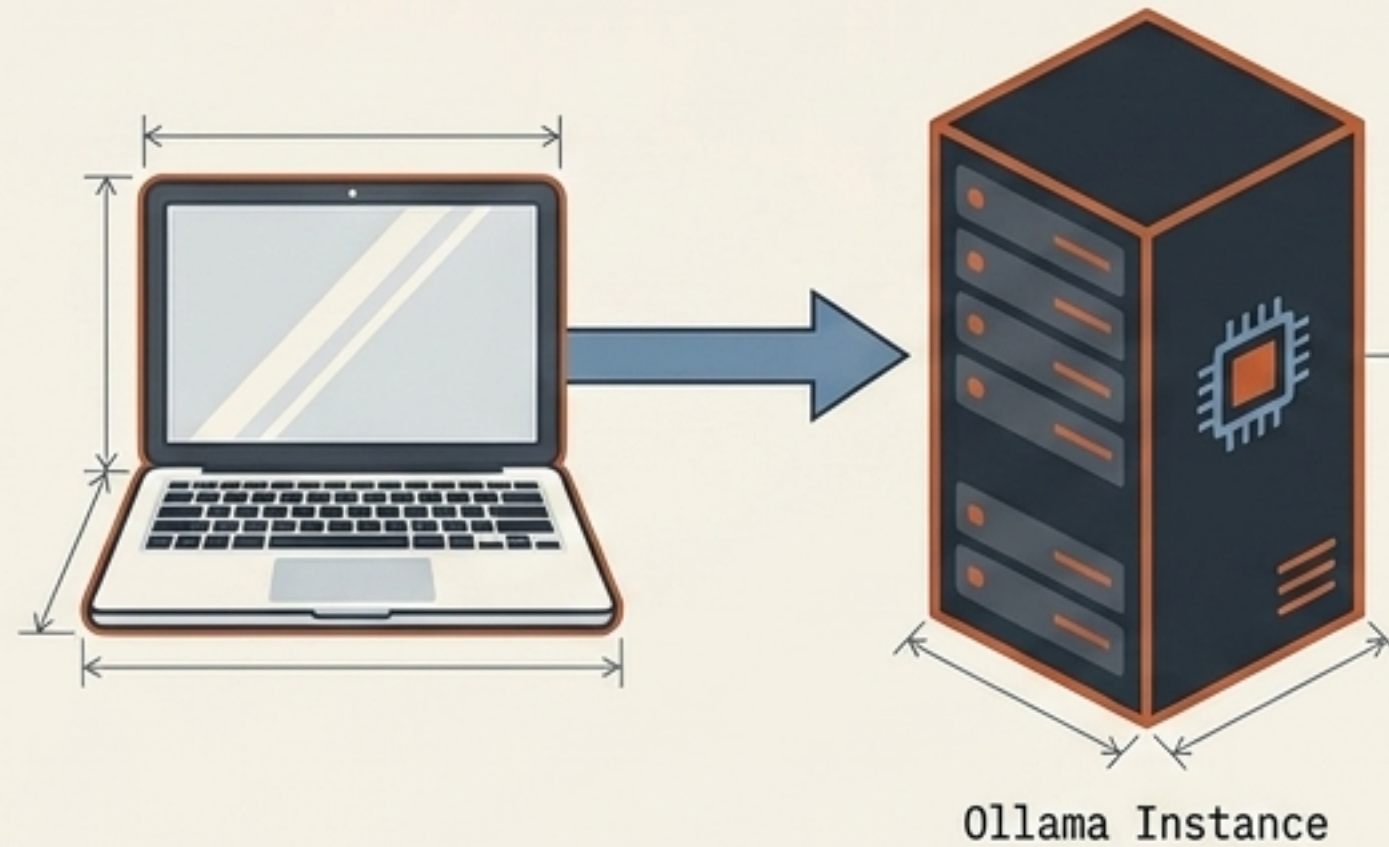
Comparison Boxes



Layer 3: High-Volume, Frugal Intelligence

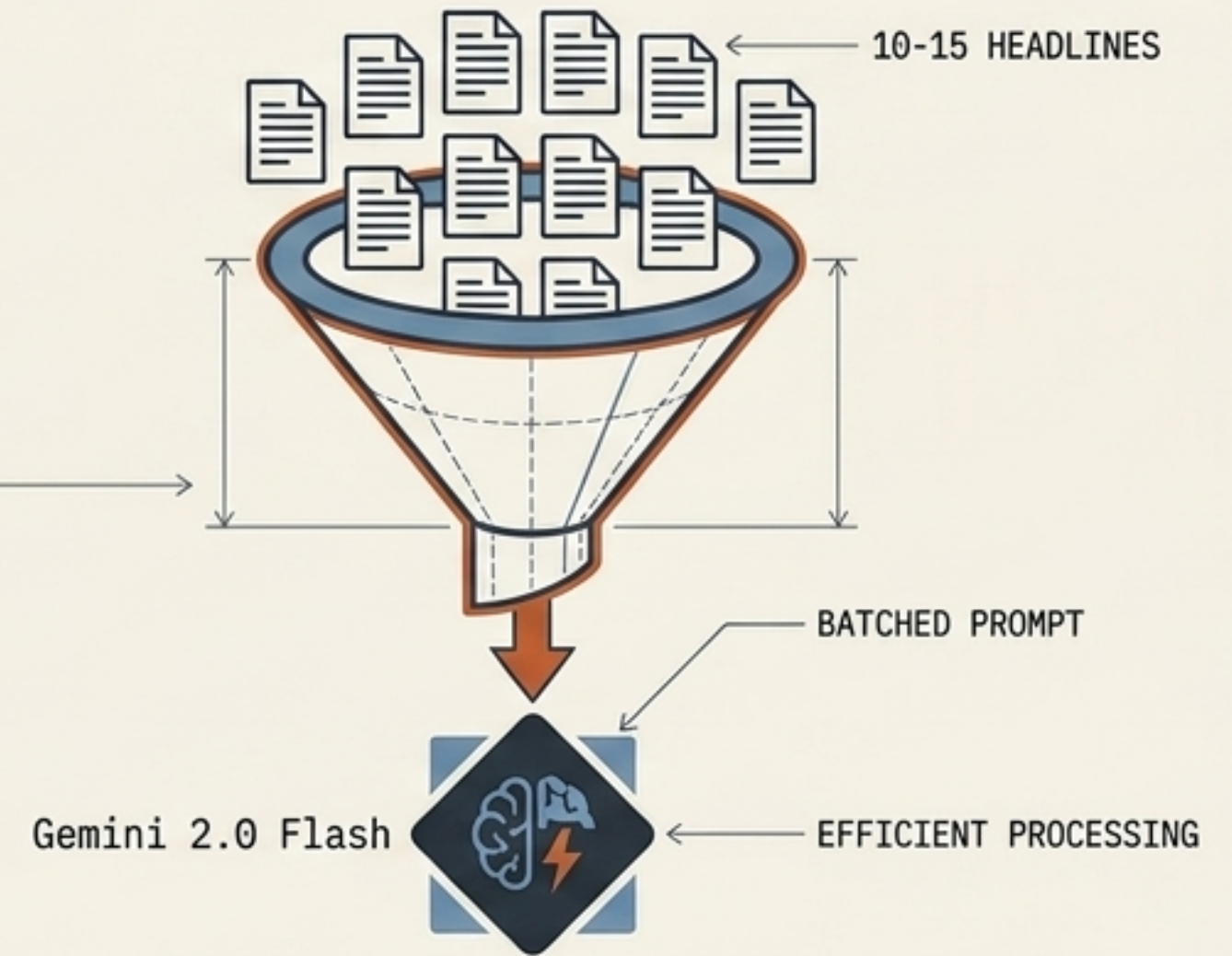


Local



Development Strategy: Use Ollama locally during the build phase to eliminate API costs entirely.

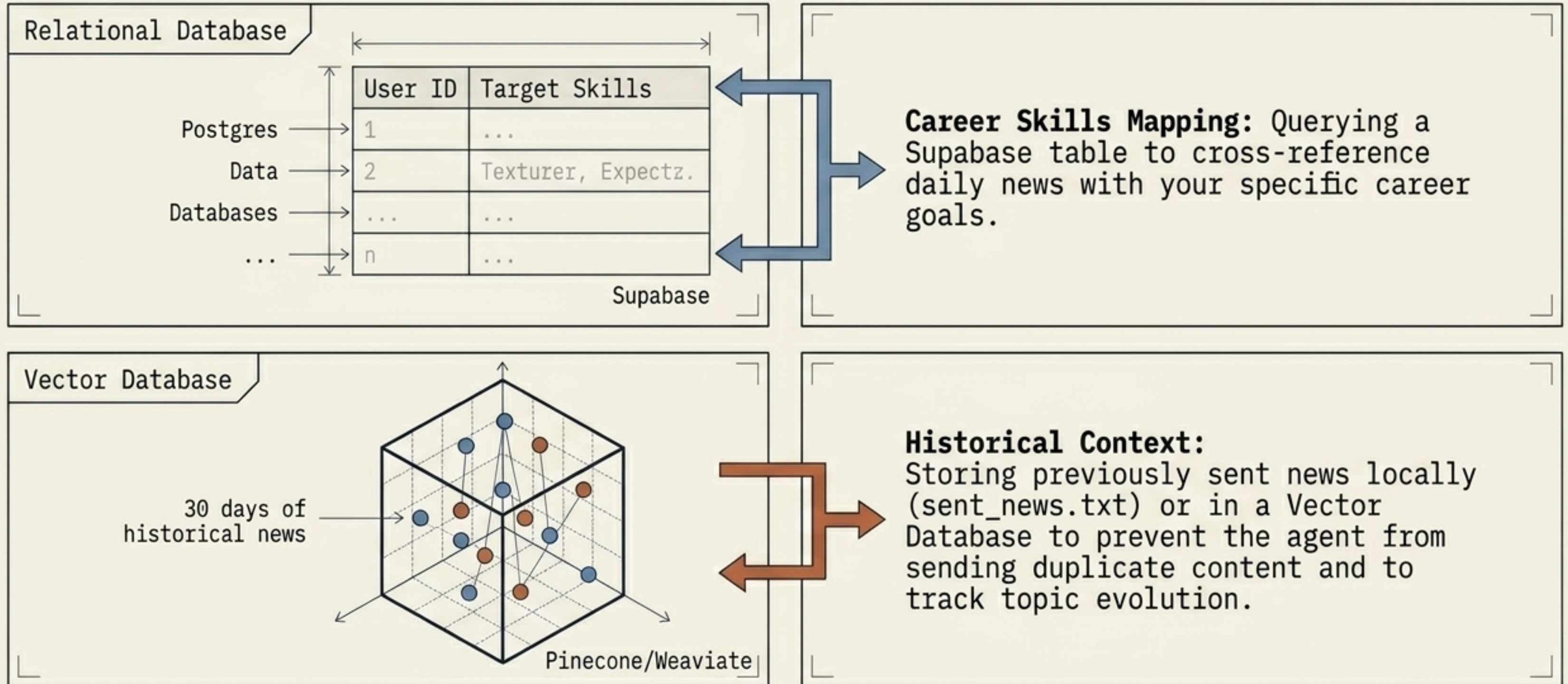
Cloud Batching



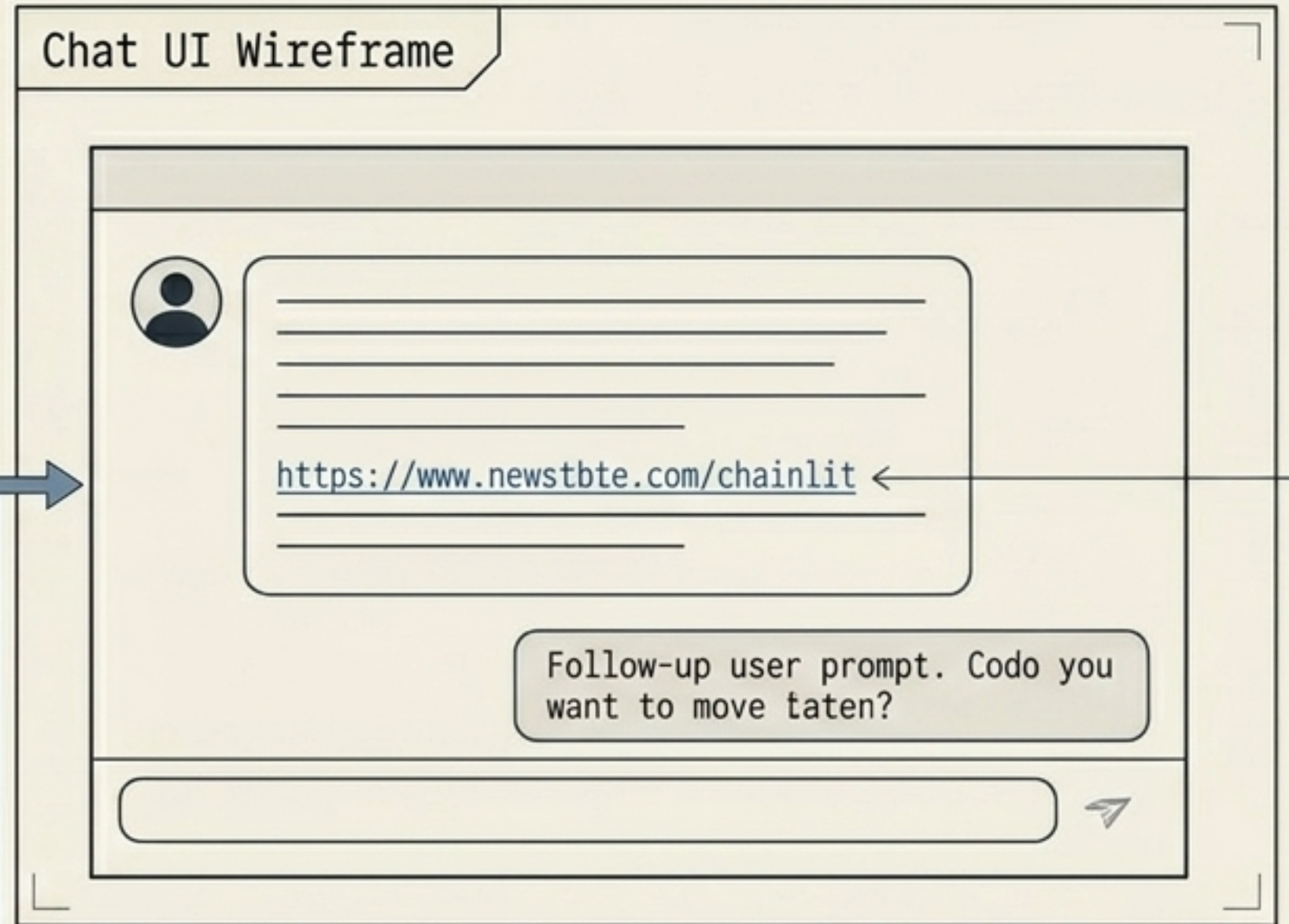
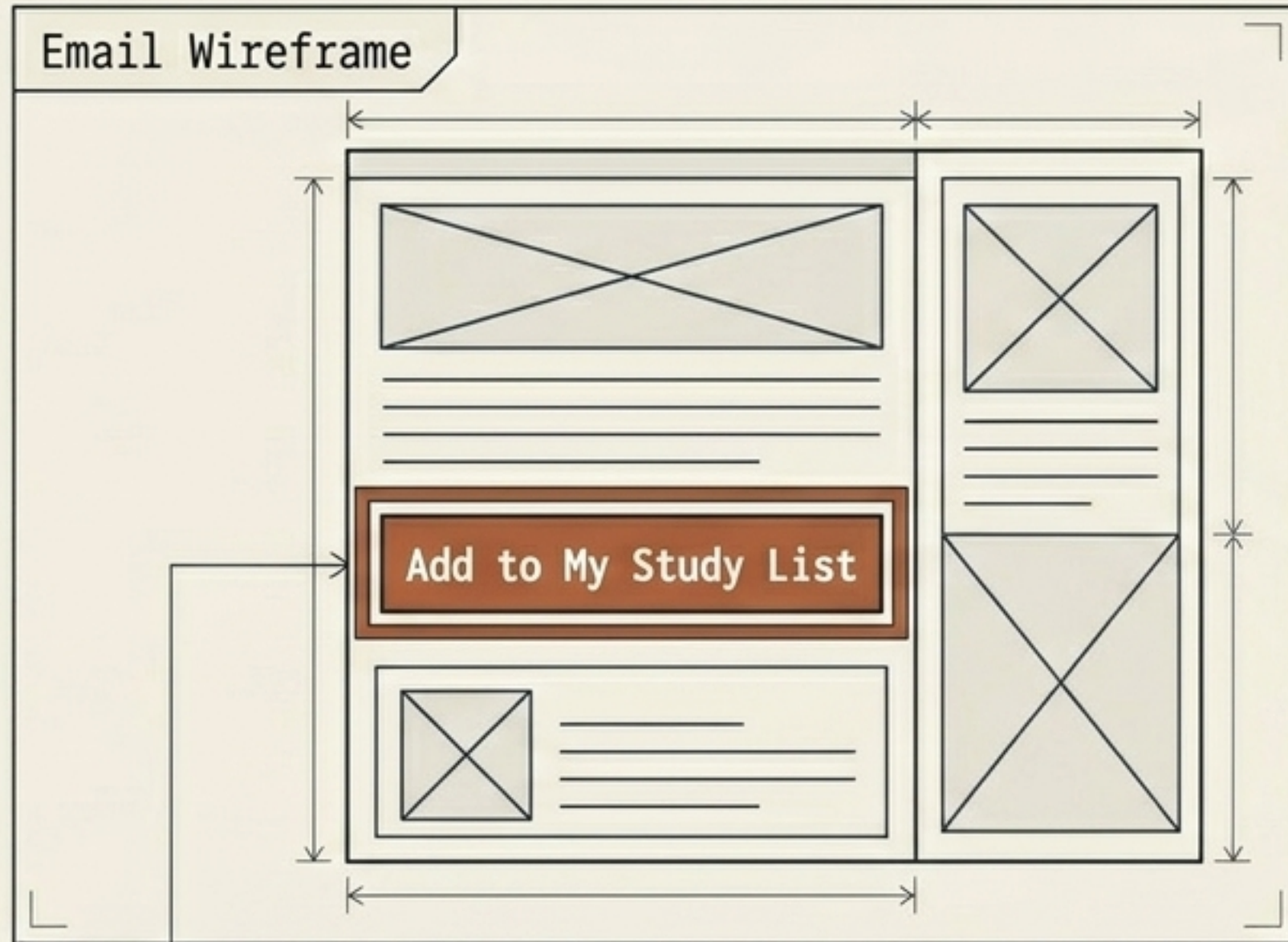
Production Strategy:
Deploy with Gemini 2.0 Flash or Grok-1.

Optimization: Implement Batch Summarization.
Send 10-15 headlines in a single prompt rather than making isolated API calls for each link.

Layer 4: State, Memory, and Deduplication



Layer 5: Interactive Outputs



Actionable Emails: Using Resend (free for 3k emails/mo) and HTML templates. Adding webhooks via embedded buttons to push tasks to Trello or Notion.

Custom Dashboarding: Utilizing Chainlit to create a dedicated, ChatGPT-style interface for the Python code to handle links, images, and buttons perfectly.

Deploying the Scheduled Workflow

on:

schedule:

- cron: '0 8 * * *'

workflow_dispatch:

Ensuring the trigger hits exactly at 8:00 AM UTC.

Enabling manual triggers for immediate testing.

jobs:

run-newsletter:

runs-on: ubuntu-latest

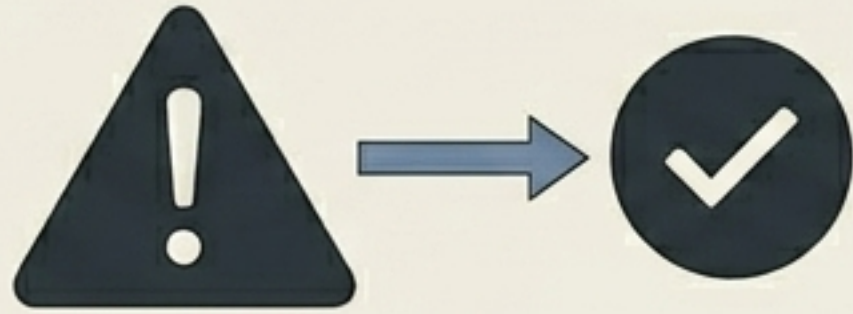
steps:

- name: Checkout code

uses: actions/checkout@v4

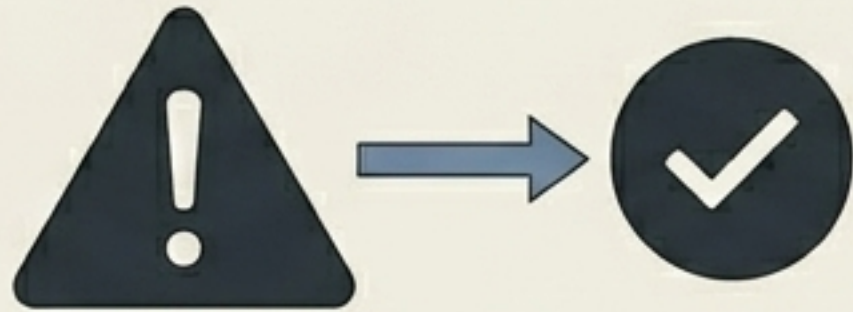
The critical step to pull the repo code into the runner environment.

Diagnostic Checklist: The 'Silent Killers' of Deployments



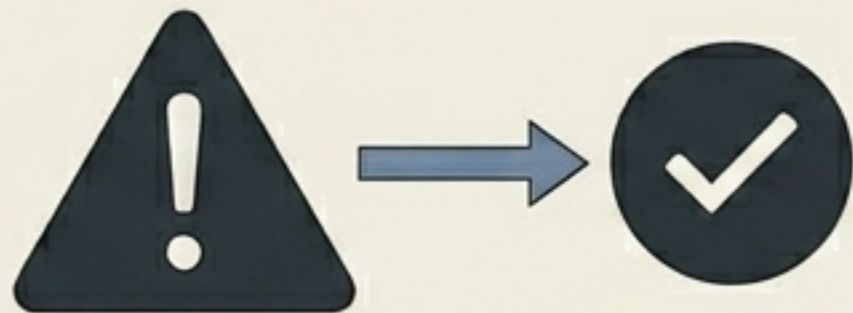
1. Environment Secrets

Scripts utilizing ``os.getenv('OPENAI_API_KEY')`` will fail unless keys are injected via **Repo Settings -> Secrets and variables -> Actions**.



2. Missing Dependencies

The absence of a ``requirements.txt`` file is the most common reason the GitHub Action environment fails to execute the Python script.

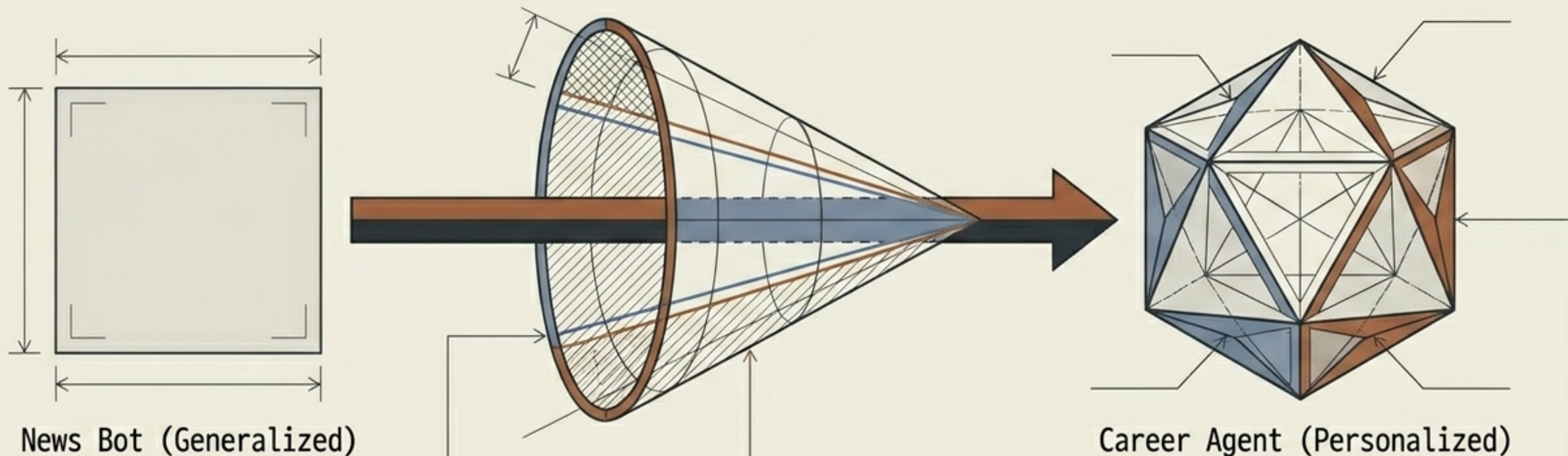


3. UTC Time Confusion

Failing to account for UTC offsets when programming the Cron schedule.



From News Bot to Career Agent



The Shift:

Moving away from generalized AI hype toward hyper-personalized intelligence.

The 'Skill of the Day':

Programming the agent to isolate a specific documentation link (e.g., 'CUDA Kernels') based on that morning's technical announcements.

The Result:

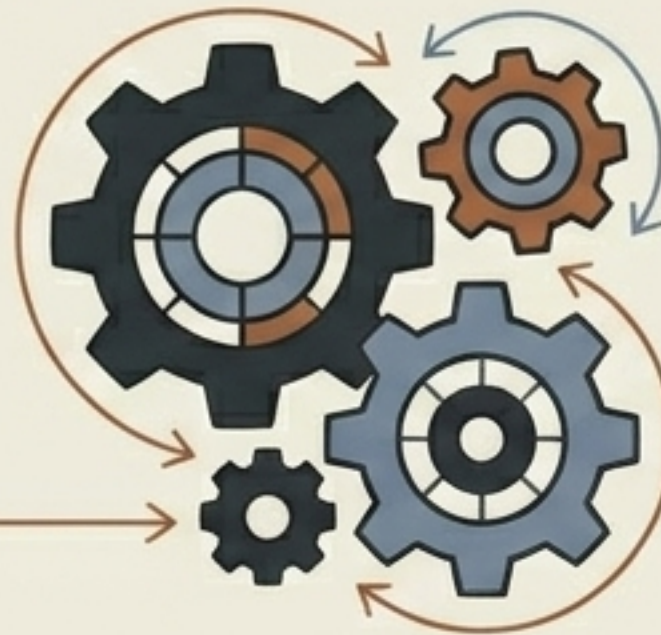
A self-contained portfolio piece that actively accelerates the creator's learning roadmap.

Baking in Enterprise-Grade Skills



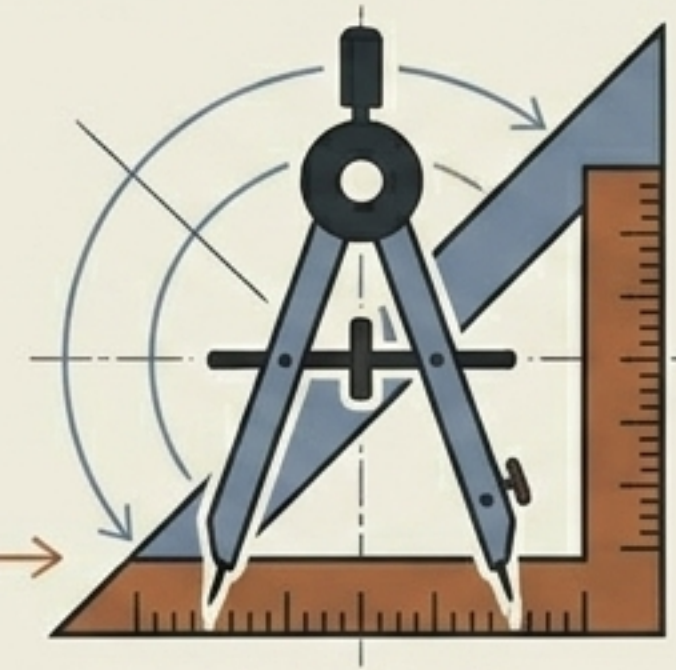
Retrieval-Augmented Generation (RAG)

→ Equipping the agent to reference historical announcements (e.g., "This is a follow-up to last week's update").



Tool Use (Function Calling)

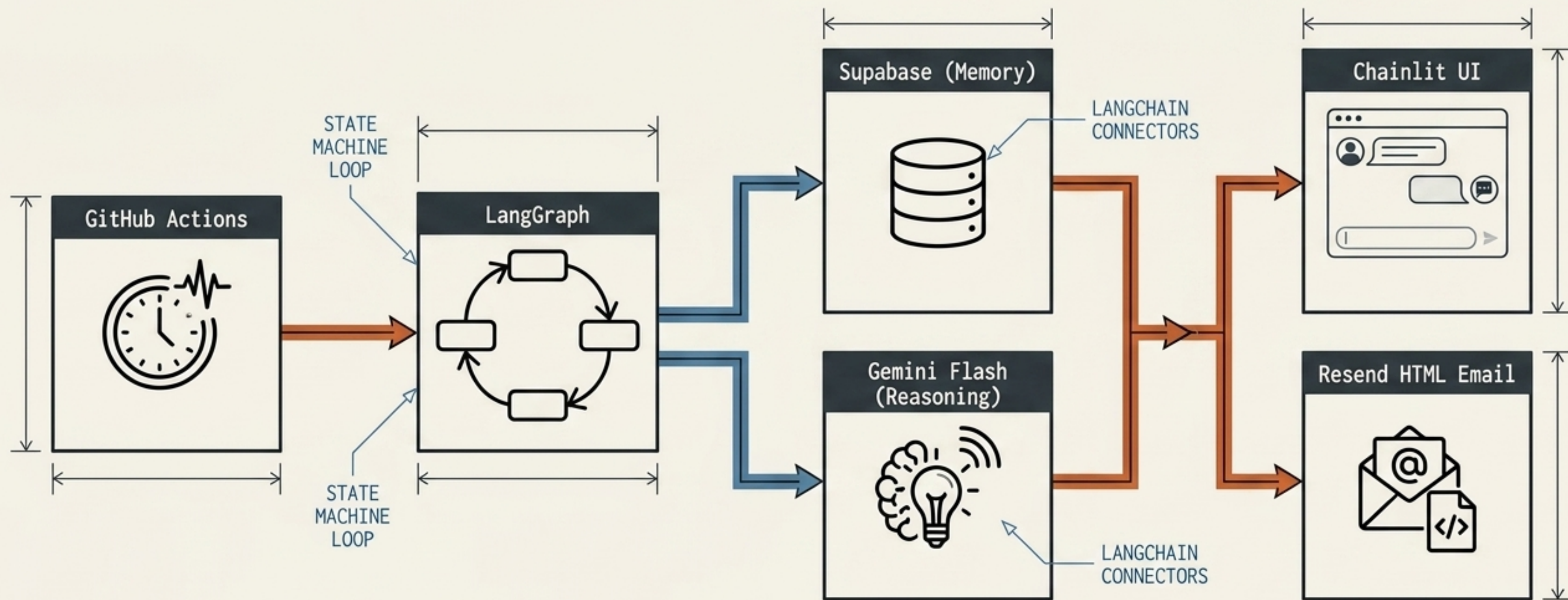
→ Moving beyond hard-coded requests by empowering the LLM to choose between `search_google()` and `scrape_website()`.



Evaluation Frameworks

→ Integrating LangSmith to empirically measure and optimize the accuracy of the agent's summaries, proving engineering rigor beyond simple prompt engineering.

The Forward Deployed Engineer Showcase



Synthesis Insight: By unifying frugal infrastructure (GitHub Actions) with stateful logic (LangGraph) and high-volume reasoning (Gemini Flash), we transform a \$0 local script into a fully autonomous, personalized career accelerator.